

IoT-BASED HVAC TEMPERATURE & HUMIDITY MONITORING SYSTEM

Submitted by

MATHESH NISHANTH B	– 24MER030
MOHAMMED SABEER S	– 24MER031
PRAAVIN RAJAA G	– 24MER039
PRASANNA T	– 24MER042
DEVA PRIYAN R	– 24MEL064
SATHISH KUMAR K	– 24MEL067



IoT-BASED HVAC TEMPERATURE & HUMIDITY MONITORING SYSTEM

1. PROJECT OVERVIEW

The IoT-Based HVAC Temperature and Humidity Monitoring System is designed to continuously monitor indoor environmental conditions and provide real-time information about temperature and humidity.

The system uses a DHT11 sensor to measure temperature and humidity. An ESP8266 Node MCU acts as the main controller and provides Wi-Fi connectivity. The measured values are displayed locally using a 16×2 I2C LCD and transmitted to the Thing Speak IoT cloud platform for remote monitoring and data storage.

The system can be used in classrooms, offices, laboratories, homes, industries, hospitals, and other buildings where maintaining suitable environmental conditions is important.

Main Objectives

- To continuously measure indoor temperature and humidity.
- To display sensor readings on an LCD.
- To transmit sensor data through Wi-Fi.
- To monitor environmental conditions remotely using ThingSpeak.
- To maintain suitable HVAC operating conditions.
- To support energy-efficient HVAC management.

2. PROBLEM STATEMENT

Traditional HVAC systems generally operate without continuously monitoring the actual environmental conditions of a particular room.

This can result in:

- Unnecessary HVAC operation.
- Excessive energy consumption.
- Poor temperature control.
- Improper humidity levels.
- Difficulty in analyzing environmental conditions over time

3. PROPOSED SOLUTION

The proposed system uses an ESP8266 NodeMCU, DHT11 temperature and humidity sensor, I2C LCD, and ThingSpeak cloud platform.

The DHT11 sensor measures the environmental conditions. The ESP8266 receives these values and displays them on the LCD. At the same time, the ESP8266 connects to Wi-Fi and uploads the readings to ThingSpeak.

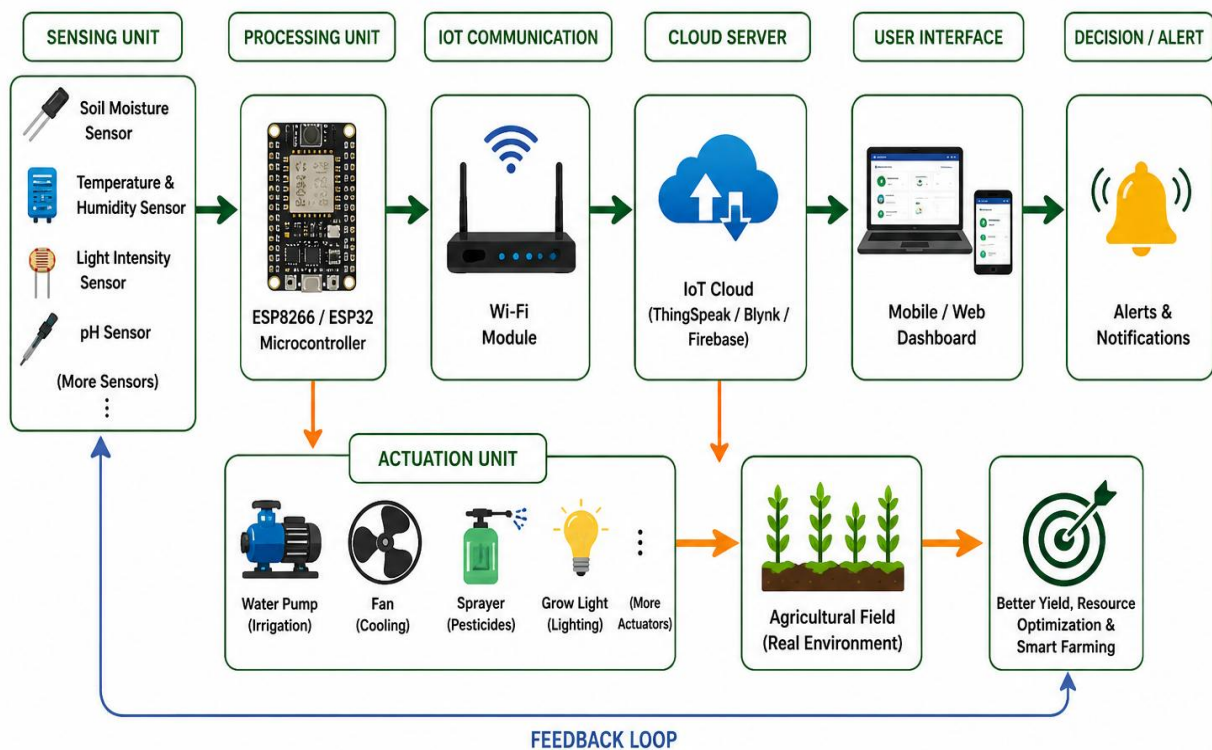
Proposed System Flow

DHT11 Sensor → ESP8266 NodeMCU → Temperature & Humidity Processing → I2C LCD Display → Wi-Fi → ThingSpeak Cloud → Remote Monitoring & HVAC Energy Management

The system provides both local monitoring and remote IoT monitoring.

4. SYSTEM ARCHITECTURE

IOT BASED SMART AGRICULTURE SYSTEM FLOW CHART



5. CIRCUIT TABLE

DHT11 to ESP8266

DHT11 Pin	ESP8266 NodeMCU
VCC	3.3V
DATA	D1
GND	GND

I2C LCD to ESP8266

I2C LCD Pin	ESP8266 NodeMCU
VCC	VIN/5V*
GND	GND
SDA	D2
SCL	D1

Component	ESP8266 Pin
DHT11 DATA	D5
LCD SDA	D2
LCD SCL	D1

Recommended Final Connection

DHT11

VCC → 3.3V

DATA → D5

GND → GND

I2C LCD

VCC → VIN/5V*

GND → GND

SDA → D2

SCL → D1

6. CIRCUIT DESIGN

The circuit is designed around the ESP8266 NodeMCU.

DHT11 Sensor

The DHT11 is connected to a digital GPIO pin of the ESP8266. It provides temperature and humidity readings to the controller.

I2C LCD

The LCD uses the I2C communication protocol. Instead of requiring many GPIO connections, only two communication lines are required:

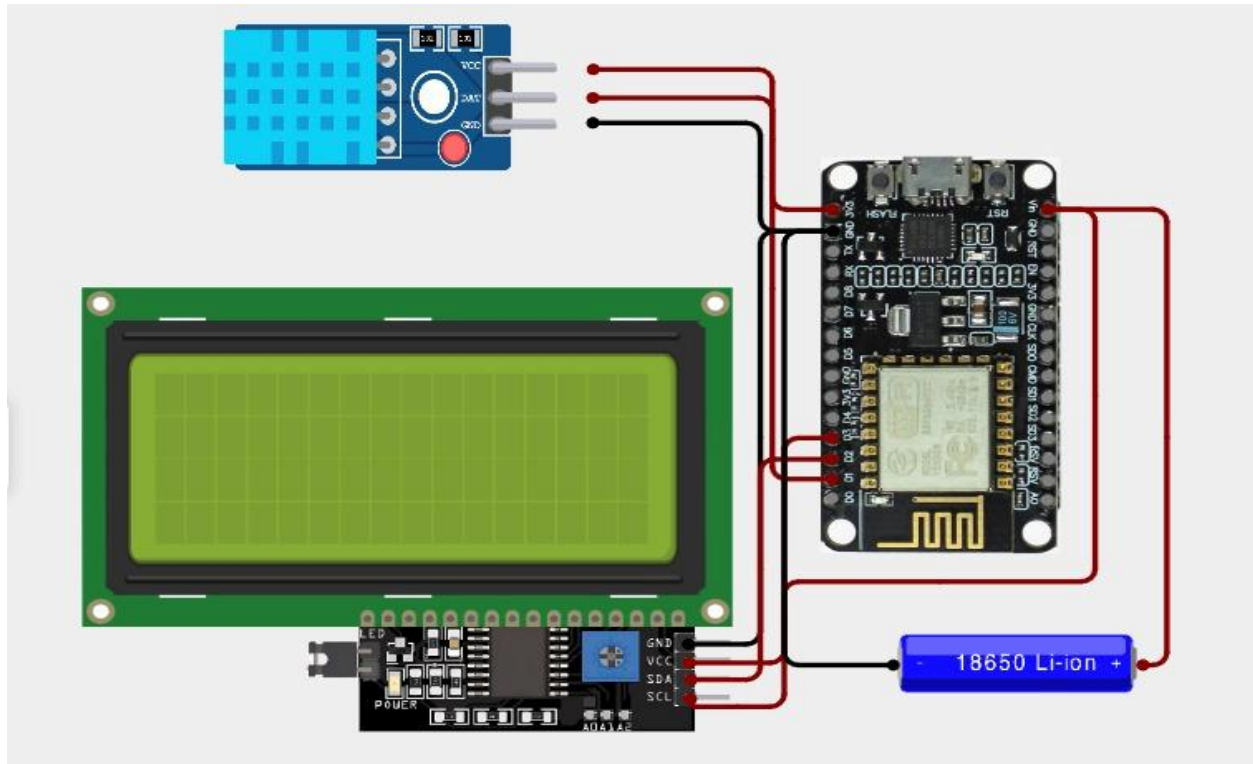
- SDA – Serial Data
- SCL – Serial Clock

ESP8266

ESP8266 performs the following operations:

1. Reads DHT11 data.
2. Processes temperature and humidity.
3. Displays values on the LCD.
4. Connects to Wi-Fi.
5. Sends values to ThingSpeak.

CIRCUIT DESIGN



7. ALGORITHM

Step-by-Step Algorithm

1. Start the system.
2. Initialize ESP8266.
3. Initialize DHT11 sensor.
4. Initialize I2C LCD.
5. Initialize serial communication.
6. Connect ESP8266 to the Wi-Fi network.
7. Read temperature from DHT11.
8. Read humidity from DHT11.
9. Check whether the readings are valid.
10. Display temperature and humidity on LCD.
11. Connect to ThingSpeak.

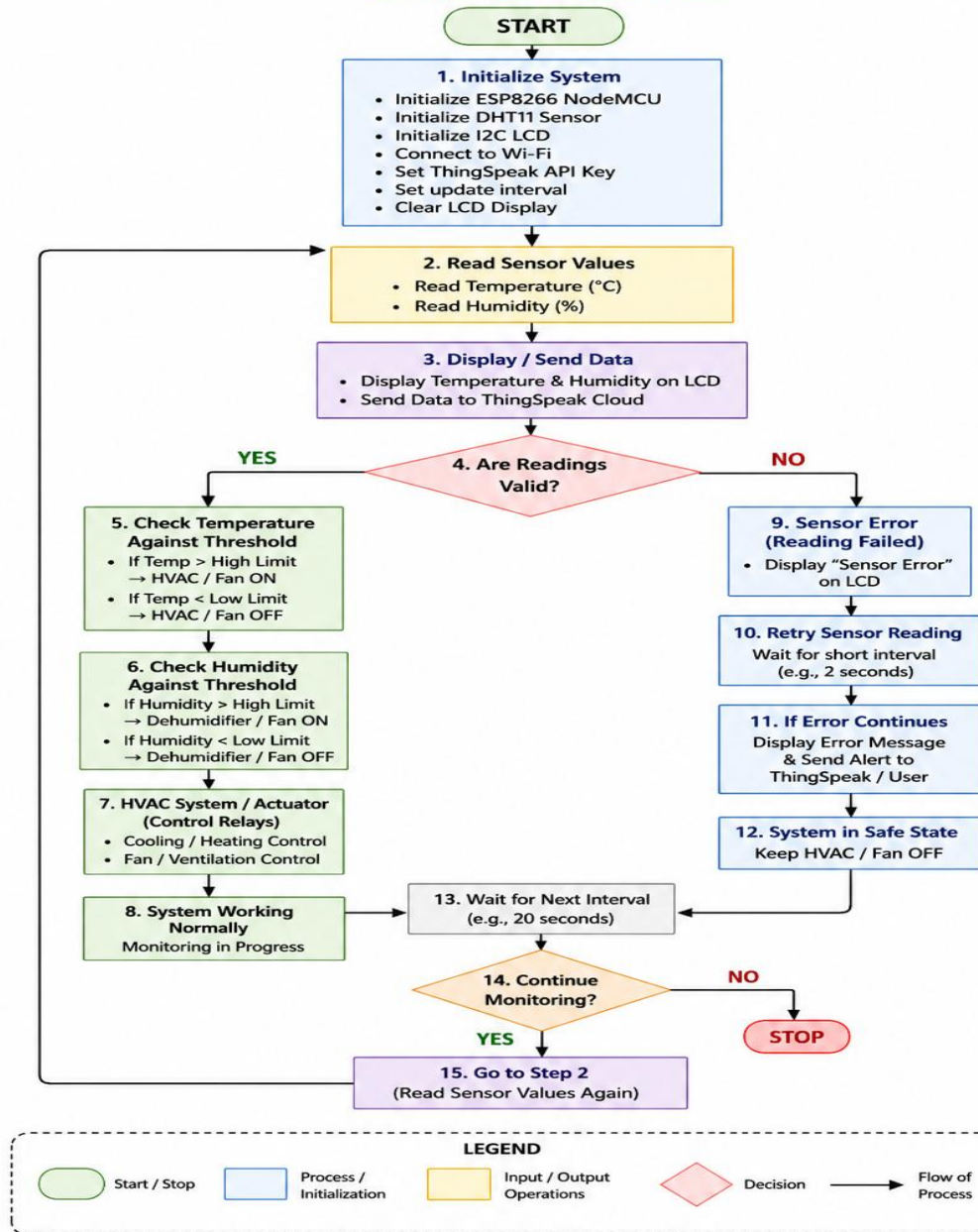
12. Upload temperature and humidity values.

13. Wait for the defined time interval.

14. Repeat the process continuously.

IOT BASED HVAC TEMPERATURE AND HUMIDITY MONITORING SYSTEM

ALGORITHM FLOWCHART



8. CODING

The following Arduino code can be used for the proposed system.

```
#include <ESP8266WiFi.h>

#include <ESP8266HTTPClient.h>

#include <DHT.h>

#include <Wire.h>

#include <LiquidCrystal_I2C.h>


const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";
const char* server = "http://api.thingspeak.com";
String apiKey = "YOUR_THINGSPEAK_WRITE_API_KEY";
#define DHTPIN D5
#define DHTTYPE DHT11


DHT dht(DHTPIN, DHTTYPE);
LiquidCrystal_I2C lcd(0x27, 16, 2);


void setup()
{
    Serial.begin(115200);


    dht.begin();

    Wire.begin(D2, D1);


    lcd.init();

    lcd.backlight();
```



```
lcd.setCursor(0, 0);  
lcd.print("HVAC MONITOR");  
lcd.setCursor(0, 1);  
lcd.print("Starting...");  
  
delay(2000);  
WiFi.begin(ssid, password);  
lcd.clear();  
lcd.setCursor(0, 0);  
lcd.print("Connecting WiFi");  
  
while (WiFi.status() != WL_CONNECTED)  
{  
    delay(500);  
    Serial.print(".");  
}  
Serial.println();  
Serial.println("WiFi Connected");  
  
lcd.clear();  
lcd.setCursor(0, 0);  
lcd.print("WiFi Connected");  
  
delay(1500);  
}  
  
void loop()
```

```
{  
    float temperature = dht.readTemperature();  
    float humidity = dht.readHumidity();  
  
    if (isnan(temperature) || isnan(humidity))  
    {  
        Serial.println("DHT11 reading failed");  
  
        lcd.clear();  
        lcd.setCursor(0, 0);  
        lcd.print("Sensor Error");  
  
        delay(2000);  
        return;  
    }  
    Serial.print("Temperature: ");  
    Serial.print(temperature);  
    Serial.println(" C");  
  
    Serial.print("Humidity: ");  
    Serial.print(humidity);  
    Serial.println(" %");  
    lcd.clear();  
  
    lcd.setCursor(0, 0);  
    lcd.print("Temp: ");  
    lcd.print(temperature, 1);
```

```
lcd.print((char)223);  
lcd.print("C");  
lcd.setCursor(0, 1);  
lcd.print("Hum: ");  
lcd.print(humidity, 1);  
lcd.print("%");  
  
if (WiFi.status() == WL_CONNECTED)  
{  
    WiFiClient client;  
    HTTPClient http;  
  
    String url = String(server) +  
        "/update?api_key=" + apiKey +  
        "&field1=" + String(temperature) +  
        "&field2=" + String(humidity);  
  
    http.begin(client, url);  
    int httpCode = http.GET();  
    Serial.print("ThingSpeak Response: ");  
    Serial.println(httpCode);  
  
    http.end();  
}  
delay(20000);  
}
```

Code Explanation

- D5 reads the temperature and humidity data from the DHT11 sensor.
- D2 is connected to the SDA pin of the I2C LCD.
- D1 is connected to the SCL pin of the I2C LCD.
- ESP8266 processes the temperature and humidity readings.
- The I2C LCD displays the measured temperature and humidity locally.
- Wi-Fi provides Internet connectivity for sending sensor data.
- ThingSpeak API Key allows the ESP8266 to upload data to the ThingSpeak cloud.
- Field 1 stores the temperature value.
- Field 2 stores the humidity value.
- The DHT11 continuously provides temperature and humidity information.
- ThingSpeak stores and displays the collected data through graphs.
- The 20-second delay controls the interval between consecutive data uploads.

ThingSpeak Fields

Configure the ThingSpeak channel as:

Field	Parameter
Field 1	Temperature (°C)
Field 2	Humidity (%)

9. System Working

When the system is powered ON, the ESP8266 initializes the DHT11 sensor, LCD, and Wi-Fi.

DHT11 continuously measures temperature and humidity.

Normal Condition

Temperature & Humidity → ESP8266 → LCD Display → ThingSpeak → Continuous Monitoring

High Temperature

High Temperature → ESP8266 → HVAC ON → Temperature Normal → HVAC OFF

High Humidity

High Humidity → ESP8266 → Dehumidifier/Fan ON → Humidity Normal → OFF

The process repeats continuously for real-time HVAC monitoring.

10. BILL OF MATERIALS

S.No	Component	Quantity	Purpose
1	ESP8266 NodeMCU	1	Main controller and Wi-Fi
2	DHT11 Sensor	1	Temperature and humidity measurement
3	16×2 I2C LCD	1	Local display
4	Breadboard	1	Circuit assembly
5	Jumper Wires	As required	Electrical connections
6	USB Cable	1	Programming and power
7	5V/USB Power Supply	1	Power source
8	Computer/Laptop	1	Programming
9	Wi-Fi Network	1	Internet connectivity
10	ThingSpeak Account	1	Cloud data monitoring

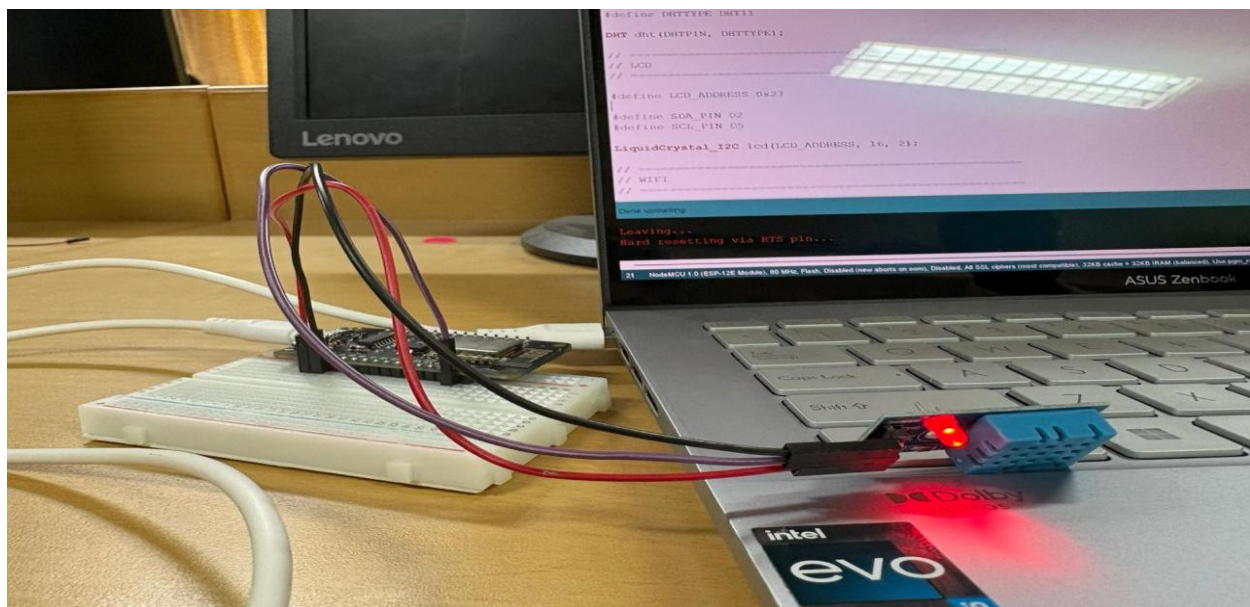
11. PROTOTYPE IMPLEMENTATION

The prototype is implemented using an ESP8266 NodeMCU, DHT11 sensor, and I2C LCD.

Implementation Steps

1. Connect the DHT11 sensor to the ESP8266.
2. Connect the I2C LCD using SDA and SCL.
3. Connect the ESP8266 to a computer using USB.
4. Install the required Arduino libraries.
5. Enter the Wi-Fi SSID and password.
6. Enter the ThingSpeak Write API Key.
7. Upload the program to ESP8266.
8. Open the Serial Monitor.
9. Verify Wi-Fi connection.
10. Observe temperature and humidity on the LCD.
11. Open the ThingSpeak channel
12. Verify that the sensor readings are uploaded successfully.

Prototype Output



12. RESULTS & PERFORMANCE ANALYSIS

The developed prototype successfully measures and monitors environmental temperature and humidity.

Sample Results

Time	Temperature	Humidity
10:00 AM	27.8 °C	61 %
10:10 AM	28.2 °C	63 %
10:20 AM	28.6 °C	64 %
10:30 AM	28.1 °C	62 %
10:40 AM	27.6 °C	60 %

Performance

The system provides the following benefits:

- Real-time temperature monitoring.
- Real-time humidity monitoring.
- Local LCD display.
- Continuous IoT monitoring.
- Remote data monitoring through ThingSpeak.
- Reduced unnecessary HVAC operation.
- Supports better energy management.

Performance Analysis

Condition	Sensor Condition	HVAC/Fan	LCD/IoT
Normal	Temperature & humidity within limit	OFF/Normal	Data displayed & uploaded
High Temperature	Temperature above required level	ON	Alert/Monitoring
High Humidity	Humidity above required level	ON	Alert/Monitoring

CONCLUSION AND WORKING IMAGE

The IoT-Based HVAC Temperature and Humidity Monitoring System was successfully designed using ESP8266, DHT11, I2C LCD, Wi-Fi, and ThingSpeak. The system provides real-time environmental monitoring, local LCD display, cloud-based data storage, and remote access to temperature and humidity information. The proposed system can be further improved by adding automatic HVAC control using a relay, temperature threshold-based control, mobile alerts, energy-consumption monitoring, and machine-learning-based predictive HVAC control. Thus, the system provides a foundation for developing a smart and energy-efficient HVAC monitoring system for homes, offices, industries, and other buildings.⁷

